

"OPERATION ANALYTICS AND INVESTIGATING METRIC SPIKE"

ADVANCED SQL

PROJECT UNDER TRAINITY

OPERATIONAL ANALYTICS

Operational Analytics is a crucial process that involves **analyzing a company's end-to-end operations**. This analysis helps identify areas for improvement within the company. As a Data Analyst, I'll work closely with various teams, such as operations, support, and marketing, helping them derive valuable insights from the data they collect.

GOAL: To use your advanced SQL skills to analyze the data and provide valuable insights that can help improve the company's operations and understand sudden changes in key metrics.

One of the key aspects of Operational Analytics is **investigating metric spikes**. This involves understanding and explaining sudden changes in key metrics, such as a dip in daily user engagement or a drop in sales. As a Data Analyst, you'll need to answer these questions daily, making it crucial to understand how to investigate these metric spikes.

TECH-STACK USED



[This Photo](#) by Unknown Author is licensed under [CC BY-SA-NC](#)



CASE STUDY 1

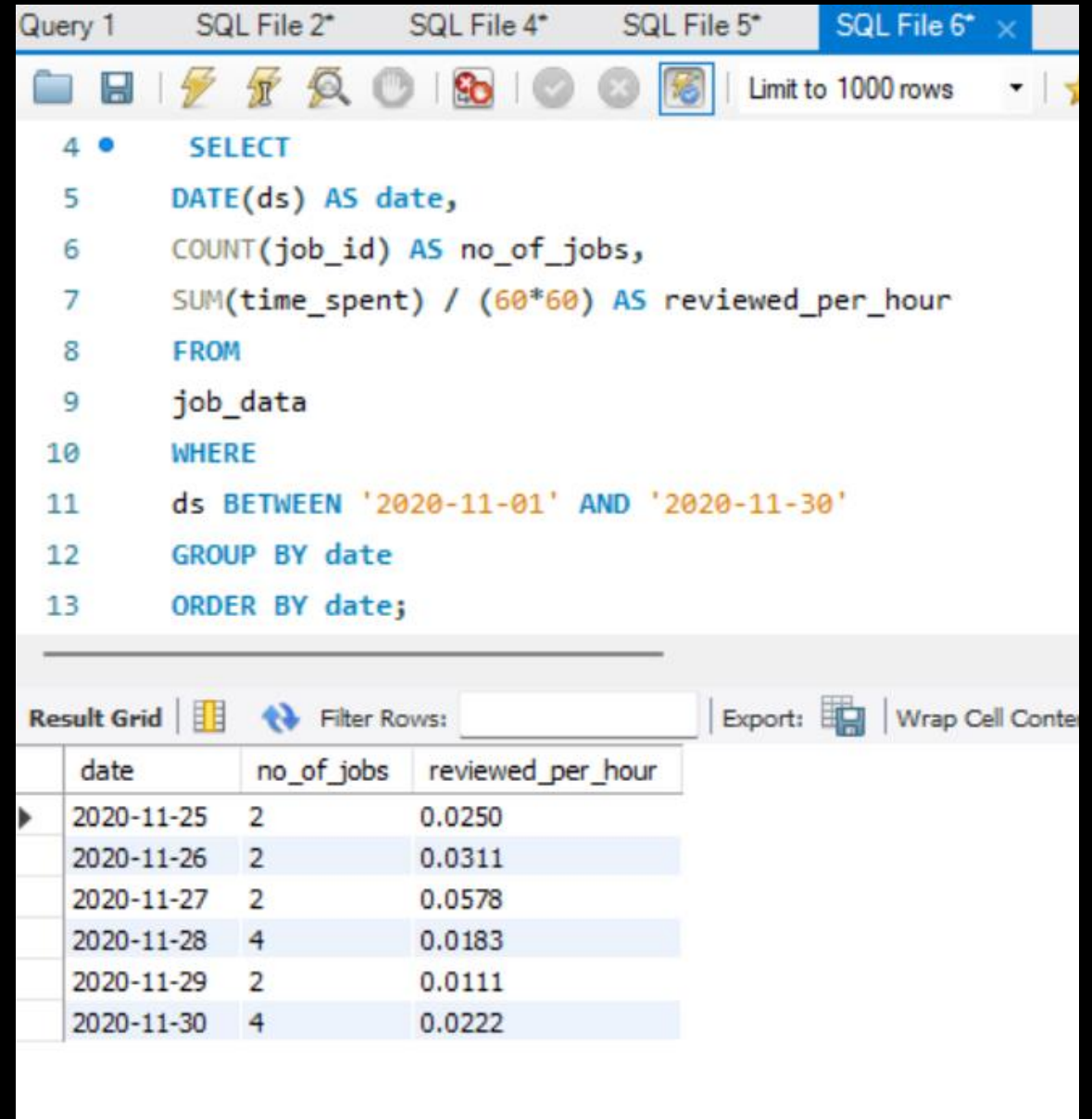
JOB DATA ANALYSIS

Jobs Reviewed Over Time

Objective: Calculate the number of jobs reviewed per hour for each day in November 2020.

MORE JOBS REVIEWED PER HOUR- 27 NOVEMBER

LESS JOBS REVIEWED PER HOUR- 29 NOVEMBER



The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```
4 • SELECT
5 DATE(ds) AS date,
6 COUNT(job_id) AS no_of_jobs,
7 SUM(time_spent) / (60*60) AS reviewed_per_hour
8 FROM
9 job_data
10 WHERE
11 ds BETWEEN '2020-11-01' AND '2020-11-30'
12 GROUP BY date
13 ORDER BY date;
```

The result grid displays the following data:

	date	no_of_jobs	reviewed_per_hour
▶	2020-11-25	2	0.0250
	2020-11-26	2	0.0311
	2020-11-27	2	0.0578
	2020-11-28	4	0.0183
	2020-11-29	2	0.0111
	2020-11-30	4	0.0222

Throughput Analysis

Objective: Calculate the 7-day rolling average of throughput (number of events per second).

The screenshot shows a SQL IDE with a query editor and a result grid. The query is as follows:

```
86  
87 • SELECT  
88     ds,  
89     COUNT(job_id) AS daily_throughput,  
90     AVG(COUNT(job_id)) OVER (  
91         ORDER BY ds  
92         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
93     ) AS rolling_7_day_avg  
94 FROM  
95     job_data  
96 GROUP BY  
97     ds  
98 ORDER BY  
99     ds;  
100
```

The result grid displays the following data:

	ds	daily_throughput	rolling_7_day_avg
▶	2020-11-25	2	2.0000
	2020-11-26	2	2.0000
	2020-11-27	2	2.0000
	2020-11-28	4	2.5000
	2020-11-29	2	2.4000
	2020-11-30	4	2.6667

Daily Metric vs. 7-Day Rolling Average

Daily Metric

PROS:

- Immediately responsive to daily fluctuations.
- It provides detailed day-by-day insights

CONS:

- Daily metrics can be volatile, especially if your data has high variability day-to-day.

7-Day Rolling Average

PROS:

- It is useful when there are seasonal variations or irregularities as it soothes out daily fluctuations.
- Better trend analysis.

CONS:

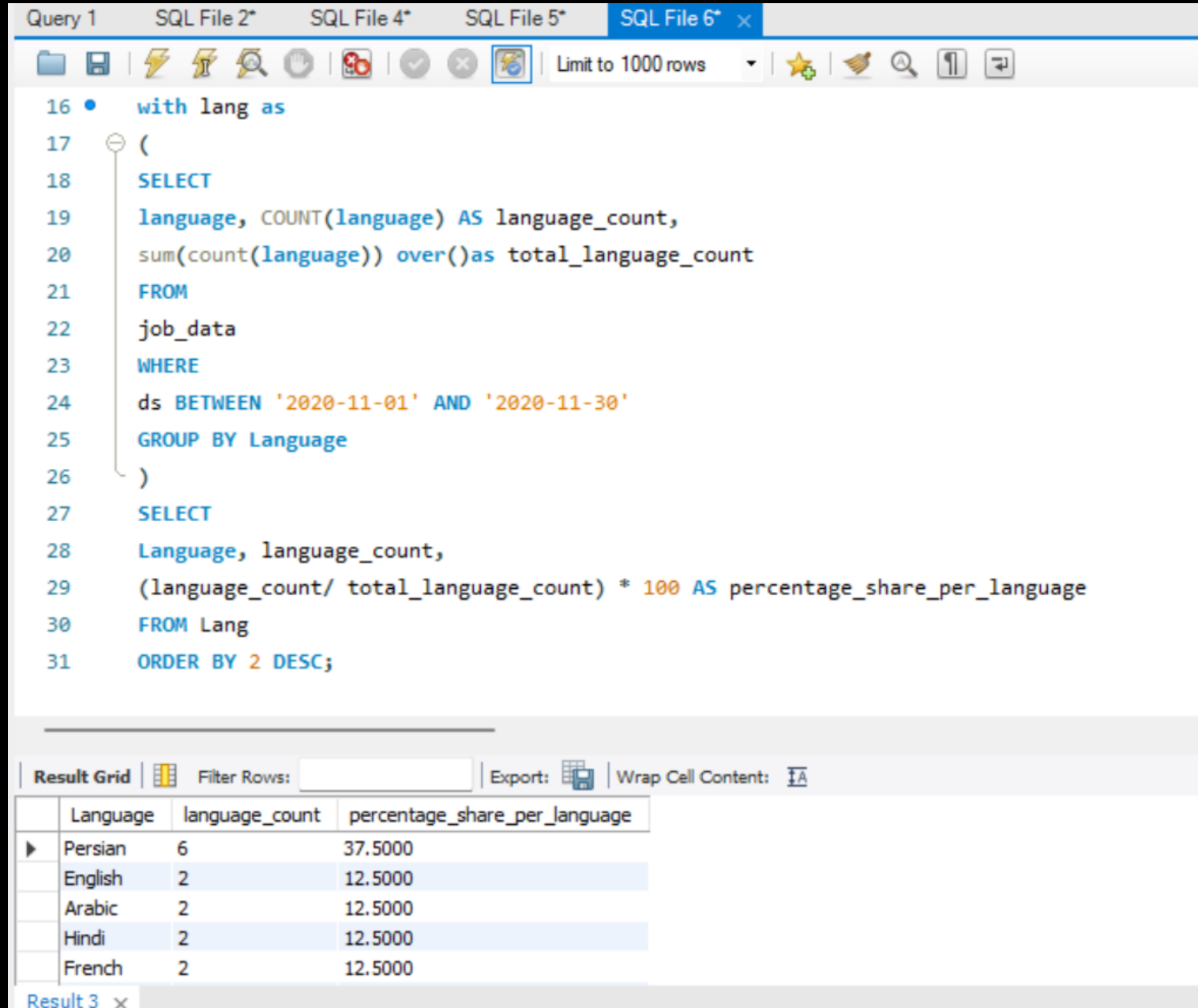
- delay the detection of sudden shifts

PREFERENCE: I generally prefer the 7-day rolling average for throughput analysis, especially when making strategic decisions or identifying trends. The rolling average smooths out the daily volatility, providing a clearer picture of long-term performance. It's particularly helpful in environments where daily operations can be erratic due to various factors like holidays, special events, or unexpected disruptions.

Language Share Analysis

Objective: Calculate the percentage share of each language in the last 30 days.

PERSIAN HOLDING HIGHEST SHARE
FOLLOWED BY HINDI



The screenshot shows a SQL IDE interface with a query editor and a results grid. The query is as follows:

```
16 • with lang as
17 (
18     SELECT
19     language, COUNT(language) AS language_count,
20     sum(count(language)) over()as total_language_count
21     FROM
22     job_data
23     WHERE
24     ds BETWEEN '2020-11-01' AND '2020-11-30'
25     GROUP BY Language
26 )
27 SELECT
28     Language, language_count,
29     (language_count/ total_language_count) * 100 AS percentage_share_per_language
30     FROM Lang
31     ORDER BY 2 DESC;
```

The results grid shows the following data:

	Language	language_count	percentage_share_per_language
▶	Persian	6	37.5000
	English	2	12.5000
	Arabic	2	12.5000
	Hindi	2	12.5000
	French	2	12.5000

Duplicate Rows Detection

Objective: Identify duplicate rows in the data.

```
126
127 • SELECT
128     *
129 FROM
130     job_data
131 WHERE
132     (actor_id, job_id, language, ds) IN (
133         SELECT
134             actor_id, job_id, language, ds
135         FROM
136             job_data
137         GROUP BY
138             actor_id, job_id, language, ds
139         HAVING
140             COUNT(*) > 1
141     );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

	ds	job_id	actor_id	event	language	time_spent	org
▶	2020-11-30	21	1001	skip	English	15	A
	2020-11-30	22	1006	transfer	Arabic	25	B
	2020-11-29	23	1003	decision	Persian	20	C
	2020-11-28	23	1005	transfer	Persian	22	D
	2020-11-28	25	1002	decision	Hindi	11	B
	2020-11-27	11	1007	decision	French	104	D
	2020-11-26	23	1004	skip	Persian	56	A
	2020-11-25	20	1003	transfer	Italian	45	C
	2020-11-30	21	1001	skip	English	15	A
	2020-11-30	22	1006	transfer	Arabic	25	B
	2020-11-29	23	1003	decision	Persian	20	C
	2020-11-28	23	1005	transfer	Persian	22	D
	2020-11-28	25	1002	decision	Hindi	11	B
	2020-11-27	11	1007	decision	French	104	D
	2020-11-26	23	1004	skip	Persian	56	A
	2020-11-25	20	1003	transfer	Italian	45	C

job_data 4 x

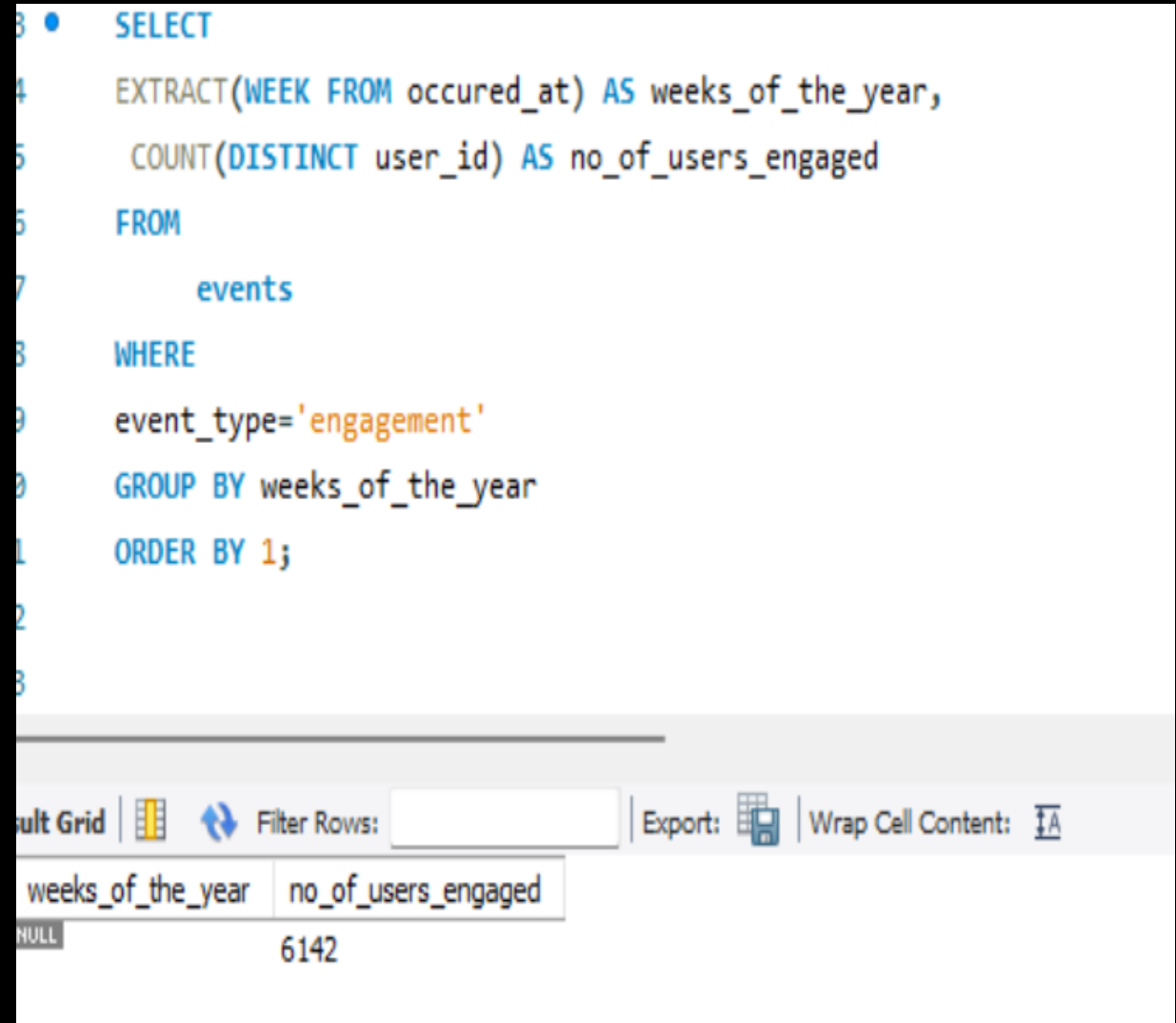
CASE STUDY 2

INVESTIGATING
METRIC SPIKE

Weekly User Engagement

Objective: Measure the activeness of users on a weekly basis.

```
3 • SELECT
4   EXTRACT(WEEK FROM occurred_at) AS weeks_of_the_year,
5   COUNT(DISTINCT user_id) AS no_of_users_engaged
6 FROM
7   events
8 WHERE
9   event_type='engagement'
10  GROUP BY weeks_of_the_year
11  ORDER BY 1;
```



weeks_of_the_year	no_of_users_engaged
NULL	6142

User Growth Analysis

Objective: Analyze the growth of users over time for a product.

```
14 SELECT year, weeknum, num_active_user,  
15 SUM(num_active_user) OVER (ORDER BY year, weeknum  
16 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cum_active_users FROM  
17 ( SELECT EXTRACT(year from a.activated_at) AS year,  
18 EXTRACT(week from a.activated_at) AS weeknum,  
19 COUNT(DISTINCT user_id) AS num_active_user  
20 FROM users a GROUP BY year, weeknum ORDER BY year, weeknum) a ;
```

Result Grid |   Filter Rows: | Export:  | Wrap Cell Content: 

year	weeknum	num_active_user	cum_active_users
NULL	NULL	9381	9381

Weekly Retention Analysis

Objective: Analyze the retention of users on a weekly basis after signing up for a product

```
22 • SELECT
23     EXTRACT(WEEK FROM occurred_at) AS sign_up_week,
24     COUNT(DISTINCT user_id) AS count_of_retention_of_users
25 FROM
26     events WHERE event_type= 'signup_flow '
27     AND event_name= 'complete_signup'
28 GROUP BY sign_up_week
29 order by 1;
```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	sign_up_week	count_of_retention_of_users		
▶	17	72		
	18	163		
	19	185		
	20	176		
	21	183		
	22	196		
	23	196		
	24	229		
	25	207		
	26	201		
	27	222		
	28	215		
	29	221		
	30	238		
	31	193		
	32	245		
	33	261		
	34	259		
	35	18		

Weekly Engagement Per Device

Objective: Measure the activeness of users on a weekly basis per device

```
32 • SELECT
33     device,
34     EXTRACT(WEEK FROM occurred_at) AS event_occurance_week,
35     COUNT(DISTINCT user_id) `weekly user engagement per device count`
36 FROM
37     EVENTS
38 WHERE
39     event_type LIKE '%engagements%'
40 GROUP BY device , event_occurance_week
41 ORDER BY 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	device	event_occurance_week	weekly user engagement per device count
	acer aspire desktop	17	9
	acer aspire desktop	18	26
	acer aspire desktop	19	23
	acer aspire desktop	20	23
	acer aspire desktop	21	29
	acer aspire desktop	22	25
Des	acer aspire desktop	23	22

Email Engagement Analysis

Objective: Analyze how users are engaging with the email service.

```
44 • select
45 (str_to_date(occurred_at,
46 '%d-%m- %Y%H:%i')) as month_name,
47 action,
48 count(user_id) as no_of_users
49 from email_events group by month_name,
50 action
51 order by action desc ;
```

month_name	action	no_of_users
2014-05-06 09:30:00	sent_weekly_digest	477
2014-05-13 09:30:00	sent_weekly_digest	491
2014-05-20 09:30:00	sent_weekly_digest	508
2014-05-27 09:30:00	sent_weekly_digest	522
2014-06-03 09:30:00	sent_weekly_digest	538

